

Digital Signal Processing Lab

Experiment 2 : LINEAR AND CIRCULAR CONVOLUTION

Group 06 (WEDNESDAY)



N DEEKSHITHA

24EC10105

SHRAVI BANSAL

24EC10130

Objectives

In this experiment, we need to perform:

- Linear Convolution
- Circular Convolution

GOAL :Take two finite-duration discrete-time signals of lengths N_1 and N_2 , respectively. In this exercise, you will compute their linear convolution (which is a finite-duration signal of length $(N_1 + N_2 - 1)$) using your fft functions. To do that, first pad zeros to both signals to ensure that both signals have length N , where N is the next power of two after $(N_1 + N_2 - 1)$. Then, compute the circular convolution of the zero-padded signals using an N -point FFT (use your fft functions for this)). You will also need to write an inverse FFT function (which can be done very easily by slightly tweaking the exponent of the twiddle factor in your fft function and adding an appropriate normalization by $(1/N)$). The circular convolution will also be an N -point signal, and the first $(N_1 + N_2 - 1)$ values of it will give you the desired linear convolution of the original signals. Check the correctness of the output computed using FFT with the naive computation of linear convolution. Compare the runtimes of the naive and the FFT-based approaches for different values of N_1 and N_2 .

CODE :

```
clc;
clear;
close all;

function y = convolution(x,h)
    N = length(x) + length(h) - 1;
    y = zeros(1,N);
    for n = 1:N
        for k = 1:length(x)
            m = n - k + 1;
            if m >= 1 && m <= length(h)
                y(n) = y(n) + x(k)*h(m);
            end
        end
    end
    return;
```

end

function [X]=FFT_Iterative(X_n,m)

N = 2^m;

rev=bitrevorder(1:N);

X=X_n(rev);

for s=1:m

M=2^s;

WM = exp((-1j)*(2*pi)/M);

for k=1:M:N

w=1;

for p=0:(M/2)-1

t=w*X(k+p+M/2);

u=X(k+p);

X(k+p)=u+t;

X(k+p+M/2) = u-t;

w = w*WM;

end

end

end

return;

end

function y_n=ifft(Y)

N= length(Y);

y_n = conj(FFT_Iterative(conj(Y),log2(N)))/N;

return;

end

function p=zero_padding(x,N)

n=length(x);

p=[x zeros(1,N-n)];

```

end

Ts=1/12000;

N1_values = [100,200,500,1e3,2e3,5e3,1e4,2e4,5e4];
N2=29;

h_n= fir1(N2-1,0.3);

for i=1:length(N1_values)
    N1=N1_values(i);
    n = 0:N1-1;
    x_n = 10*cos(2*pi*1e3*n*Ts)+6*cos(2*pi*1e3*2*n*Ts)+2*cos(2*pi*1e3*4*n*Ts);

    tic;
    y_lin_naive = convolution(x_n,h_n);
    t_lin_naive(i) = toc;

    N=2^nextpow2(N1+N2-1);

    signal = zero_padding(x_n,N);
    filter = zero_padding(h_n,N);

    X = FFT_Iterative(signal, log2(N));
    H = FFT_Iterative(filter, log2(N));

    Y=X.*H;

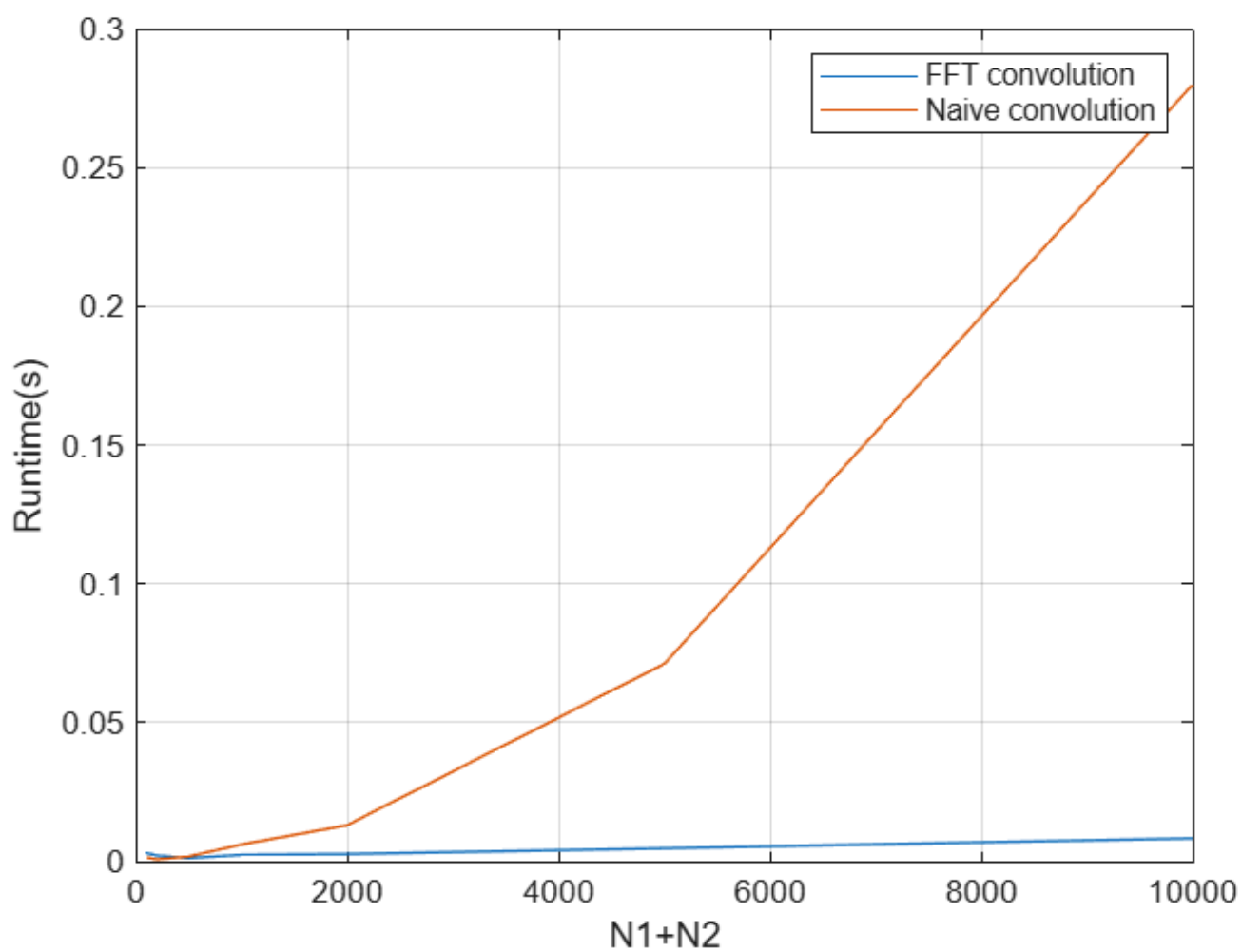
    tic;
    y_cir=real(ifft(Y));
    y_lin_fft = y_cir(1:(N1+N2-1));
    t_lin_fft(i) = toc;

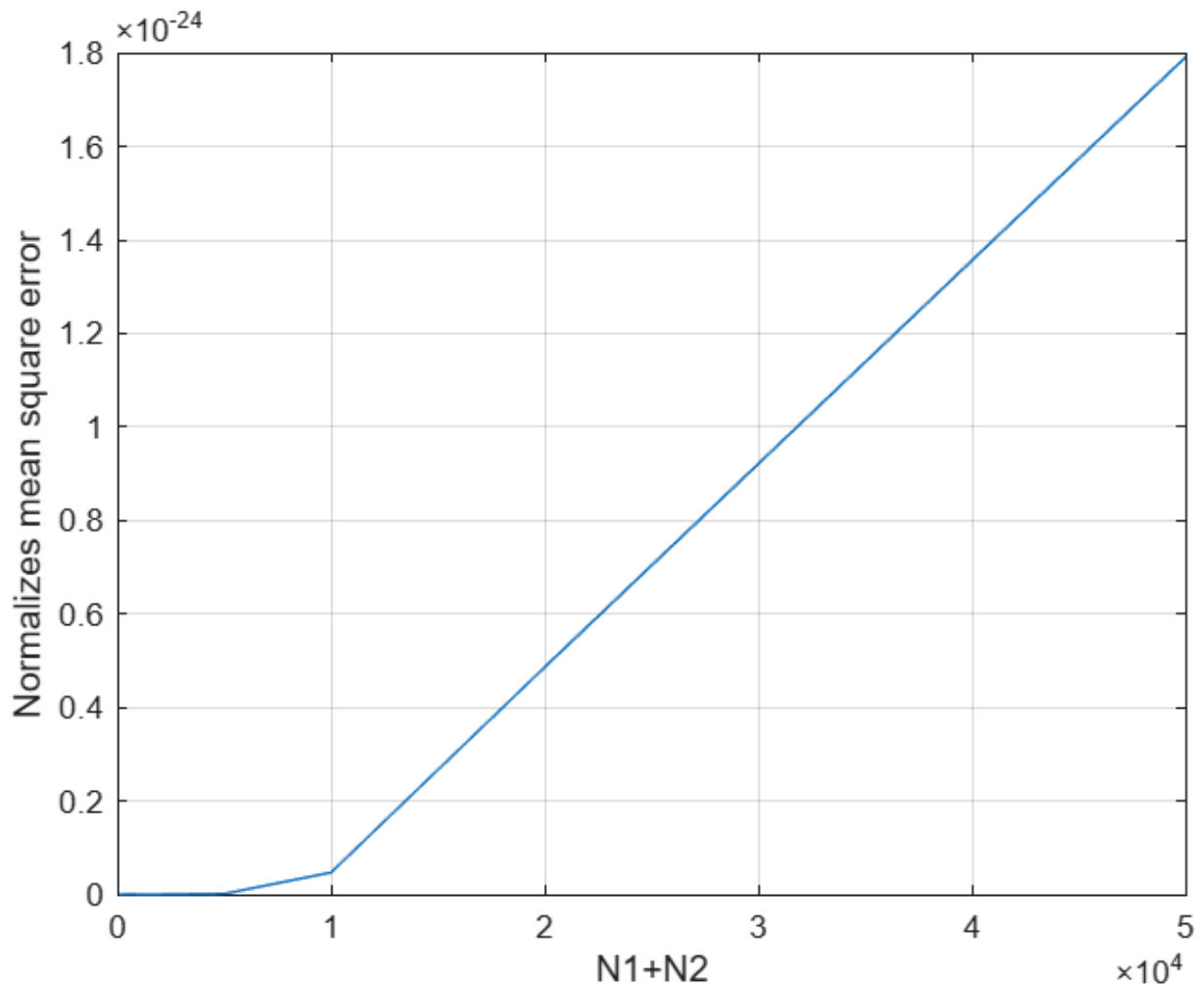
    e(i) = sum( (abs(y_lin_fft - y_lin_naive)).^2 ) / sum( (abs(y_lin_naive)).^2 );
end

figure;
plot(N1_values,t_lin_fft); hold on;
plot(N1_values,t_lin_naive);

```

```
legend('FFT convolution','Naive convolution');grid on;  
xlabel('N1+N2');  
  
ylabel('Runtime(s)');  
figure;  
  
plot(N1_values,e);grid on;  
xlabel('N1+N2');  
ylabel('Normalizes mean square error');
```





RESULT :

- For small input sizes (e.g., $N=100,200$), the direct convolution (naïve method) was faster or comparable to FFT-based convolution, because the FFT introduces extra overhead due to zero-padding and recursive butterfly operations.

$N = 100$ Runtime_Naive = 0.0013 Runtime_FFT = 0.0031

$N = 200$ Runtime_Naive = 0.0007 Runtime_FFT = 0.002

- For small sizes ($N \leq 300$), the naïve runtime was lower due to less overhead.
- Around $N=1000$, the runtimes started crossing.
- By $N \approx 10,000$, the **naïve runtime grew much larger than FFT-based runtime**, proving the asymptotic advantage of FFT convolution.
- The **Normalized Mean Square Error (NMSE)** between naïve and FFT-based convolution was extremely small (close to $1e-25$).

DISCUSSION :

N DEEKSHITHA (24EC10105) :

- The experiment validates the
 - Direct convolution is simple and efficient for small N .
 - FFT - based convolution becomes much faster for large signals $O(k \log k)$ scaling.
- Zero-padding ensures that circular convolution matches linear convolution. Without padding, aliasing would distort the result.
- The choice of power-of-2 FFT sizes (radix-2) further improves efficiency, since the butterfly structure is simple and optimized. Although other radices (radix-4, radix-8, etc.) are possible, radix-2 is most commonly used due to hardware or software support.
- This matches theoretical complexity:
 - **Naive convolution:** $O(N_1 \cdot N_2)$
 - **FFT convolution:** $O(k \log k)$, where $k \geq N_1 + N_2 - 1$, usually padded to the next power of 2.
- **Error remained negligible** across all experiments, confirming correctness of both approaches.

SHRAVI BANSAL (24EC10130) :

- Linear convolution gives the system's output for any input, while circular convolution enables FFT-based computation, crucial for speeding up processing for long signals using the Fast Fourier Transform.
- Both signals are zero-padded to length N , where N is the next power of two after $N_1 + N_2 - 1$ (signal and filter lengths).

Reason: Padding to the next power of two ensures FFT algorithm efficiency (particularly radix-2 FFT), avoids aliasing in circular convolution.

- Convolution in time domain equals multiplication in frequency domain. FFT reduces complexity from $O(N^2)$ to $O(N \log N)$. Zero-padding ensures that circular convolution mimics linear convolution.
- For small signals, overhead of FFT makes direct convolution faster; for large signals, FFT is much more efficient due to better scaling $O(N \log N)$ vs $O(N_1 N_2)$.

IN GRAPH:

Runtime Comparison Graph

- For **small inputs** (e.g., $N=100,200$), the naive method is slightly faster, as FFT involves overhead from zero-padding and internal operations (bit-reversal, butterfly stages). This matches complexity theory: naive convolution needs only basic arithmetic for small arrays.
- As the input size increases (beyond $N=1000$), the runtime for naive convolution rapidly grows, while FFT-based convolution scales much better. Eventually, FFT is far more efficient for large signals ($N \approx 10,000$ and above), due to its lower complexity of $O(k \log k)$ compared to naive's $O(N_1 N_2)$.
- **Key takeaway:** The vertical gap between the two curves at high N quantifies the substantial computational savings provided by FFT convolution.

Normalized Mean Square Error (NMSE) Graph

- Across all tested signal sizes, the NMSE values remain extremely low (on the order of 10^{-25}), indicating the two algorithms produce virtually identical outputs for linear convolution, modulo negligible numerical round-off differences.
- The stability of the NMSE curve, regardless of input size, confirms the **correctness and reliability** of FFT-based convolution when zero-padding is correctly applied.